# HW #9.

Prob. 1. Solve

(1) $\frac{dy}{dx} = y - x.$

(2) $y(0) = 1.$ using the method:

(3) $y_{m+1} + \frac{3}{2} y_m - 3y_{m-1} + \frac{1}{2} y_{m-2} = 3h\, f_m.$

This is the exact problem with previous HW #8 Problem 2 I think. So, I'll be uploading the solution from it on the next starting page:

Prob. 2. Solve

(1) $\frac{dy}{dx} = y - x.$

(2) $y(0) = 1.$ using the ↓ method

(3) $y_{i+1} + \frac{3}{2} y_i - 3 y_{i-1} + \frac{1}{2} y_{i-2} = 3h f_i$

Sol-n.

(1) $\frac{dy}{dx} = y - x$

$y' - y = -x. \quad \Big| \cdot e^{-x}$

$y' \cdot e^{-x} - y \cdot e^{-x} = -x e^{-x}$

$\frac{d}{dx}\left(y \cdot e^{-x}\right) = -x \cdot e^{-x}$

$\int d\left(y \cdot e^{-x}\right) = \int -x e^{-x} dx + C.$

$y e^{-x} = \int \underbrace{x}_{u} d\underbrace{(e^{-x})}_{v} + C \Rightarrow$

$y \cdot e^{-x} = x e^{-x} - \int e^{-x} dx + C.$

$y \cdot e^{-x} = x e^{-x} + e^{-x} + C. \quad \Big| \cdot e^{x}$

(4) $\boxed{y = x + 1 + C \cdot e^{x}}$ using the IV in (2):

$y(0) = 1 = 0 + 1 + c \cdot e$

$ce^0 = 0 \Rightarrow \boxed{c = 0}$

(6) $\boxed{y(x) = x + 1}$

Note the solution exists because

(7) $f(x,y) = y - x \in C(\mathbb{R}^2) =$ cont-us on $\mathbb{R}^2$.

and unique because

(8) $\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(y - x) = 1 \in C(\mathbb{R}^2) =$ con-t on $\mathbb{R}^2$

Using the L-M method in (3):

(9) $y_{i+1} = \frac{-3y_i}{2} + 3y_{i-1} - \frac{y_{i-2}}{2} + 3h \cdot f(x_i, y_i)$

$\Rightarrow$

(10) $y_{i+1} = \frac{-3y_i}{2} + 3h(y_i - x_i) + 3y_{i-1} - \frac{y_{i-2}}{2}$

say $\boxed{i = 2}$

(11) $y_3 = \frac{-3y_2}{2} + 3h(\overset{?}{y_2 - x_2}) + 3\overset{?}{y_1} - \frac{y_0}{2}$. ✓

$x_0$ $y_0$ $x_1$ $y_1$ $x_2$ $y_2$ Exact $y_2$



To find $y_3$ we need extra $y_1, x_2, y_2$ values which we dont know. For the we are allowed to use the exact sol-n (6).

(6) $\boxed{y(x) = x+1}$ $x_0 = 0$,

(12) $\begin{cases} x_1 = x_0 + h. \\ y_1 = y(x_1) = x_1 + 1 \end{cases}$

But to work on (12) we need $h$!

Since (3) is explicit L.M. method, it does have cond-n on $h$, which we provided in Pr.1.

(13) $h < \frac{2}{\lambda}$, $\lambda = \max\left\{ \left|\frac{\partial f}{\partial y}\right| ; (x,y) \in D(f) \right\}$

$y(7)$

$\lambda = \max\left\{ \left|\frac{\partial}{\partial y}(y-x)\right| ; (x,y) \in \mathbb{R}^2 \right\} = 1.$

So $h < \frac{2}{\lambda} = \frac{2}{1} = 2 \Rightarrow$

(14) $\boxed{h < 2}$.

To check how choice of $h$ influences the

numerical sol-s, I'm going to write a code for

$$h = \frac{1}{2^3}, \frac{1}{2^2}, \frac{1}{2} < 2.$$

...

Problem 2. Use Picard's iteration to solve:

(i) $\frac{dy}{dx} = xy$ (1)

(2) $y(0) = 1 \qquad x \in [0,1]$

Solution:

$$dy = xy\,dx \quad \Big\{ \text{BS} \int_{x_0}^{x} \ldots ds \quad \Rightarrow$$

$$\int_{x_0}^{x} dy = \int_{x_0}^{x} s y\, ds$$

(3) $$\boxed{y = y_0 + \int_{x_0}^{x} s y\, ds}$$ Picard Iteration formula:

$$y_1 = y_0 + \int_{x_0}^{x} s y_0\, ds$$

$$y_2 = y_0 + \int_{x_0}^{x} s y_1\, ds$$

$$y_3 = y_0 + \int_{x_0}^{x} s y_2\, ds$$

$$\ldots$$

So the arithmetics are

$$y_1 = y_0 + \int_{x_0}^{x} s y_0 \, ds = 1 + \int_{x_0=0}^{x} s \, ds = 1 + \frac{s^2}{2}\Big|_{x_0=0}^{x} = 1 + \frac{x^2}{2}$$

$$y_2 = 1 + \int_0^x s \cdot \left(1 + \frac{s^2}{2}\right) ds = 1 + \int_0^x s + \frac{s^3}{2} \, ds = 1 + \frac{s^2}{2} + \frac{s^{3+1}}{2\cdot(3+1)}\Big|_0^x$$

$$= 1 + \frac{x^2}{2} + \frac{x^4}{2\cdot 4}$$

$$y_3 = 1 + \int_0^x s \cdot \left(1 + \frac{s^2}{2} + \frac{s^4}{2\cdot 4}\right) ds = 1 + \frac{s^2}{2} + \frac{s^4}{2\cdot 4} + \frac{s^6}{2\cdot 4\cdot 6}\Big|_0^x =$$

$$= 1 + \frac{x^2}{2} + \frac{x^6}{2\cdot 4\cdot 6} + \frac{x^4}{2\cdot 4},$$ So by pattern

$$y_4 = 1 + \frac{x^2}{2} + \frac{x^4}{2\cdot 4} + \frac{x^6}{2\cdot 4\cdot 6} + \frac{x^8}{2\cdot 4\cdot 6\cdot 8}$$ & so

$$y_n = 1 + \frac{x^2}{2} + \frac{x^4}{2\cdot 4} + \frac{x^6}{2\cdot 4\cdot 6} + \dots + \frac{(x^2)^n}{2^n \cdot n!} = \frac{\left(\frac{x^2}{2}\right)^n}{n!} \approx e^{\frac{x^2}{2}}$$

So $y(x) = e^{\frac{x^2}{2}}$.

(ii)

(1) $\frac{dy}{dx} = 2(y+1)$

(2) $y(0) = 0 \qquad x \in [0,1].$

Sol-n:

$dy = (2(y+1)\,dx \qquad \int_{x_0=0}^{x} \ldots ds$ of BS:

$$\int_0^x dy = \int_0^x 2(y+1)\,ds$$

(3) $\boxed{y(x) = y_0 + \int_0^x 2(y+1)\,ds}$ So

$$y_1 = y_0 + \int_0^x 2(y_0+1)\,ds = 0 + \int_0^x 2\,ds = 2x$$

$$y_2 = y_0 + \int_0^x 2(y_1+1)\,ds = 0 + \int_0^x 2(2s+1)\,ds = 2x^2 + 2x$$

$$y_3 = \int_0^x 2\cdot(2s^2+2s+1)\,ds = 2\cdot 2\cdot \frac{s^3}{3}\Big|_0^x + 2s^2\Big|_0^x + 2s\Big|_0^x$$

$$= 2x^2 + \frac{4}{3}x^3 + 2x$$

$$y_4 = \int_0^x 2\left(2s^2 + \frac{4}{3}s^3 + 1\right)ds = 2\left(s^2 + \frac{2s^3}{3} + \frac{4}{3\cdot 4}s^4 + s\right)\Big|_0^x$$

$$= 2\left(x + x^2 + \frac{2x^3}{3} + \frac{1}{3}x^4\right) \quad \ldots$$

I, to be honest, could not fell the pattern for picking the Taylor series, so I stopped at $y_4$. But I believe this is the process for the Picard's iteration.

Prob 3. Exact discrete solution of the IVP

(1) $y' = -5y$

(2) $y(0) = 1$

3 points given so the given Runge Kutta method is order 2.

Exact sol-n:

$$\frac{dy}{dx} = -5y$$

$$\int \frac{dy}{y} = \int -5 \, dx + \ln C.$$

$$\ln|y| = \ln e^{-5x} + \ln C \Rightarrow \ln|y| = \ln(e^{-5x} \cdot C)$$

$$y = e^{-5x} \cdot C \overset{IV}{\Rightarrow} y(0) = 1 = e^{-5 \cdot 0} \cdot C \Rightarrow \boxed{C = 1}$$

So,

(3) $\boxed{y(x) = e^{-5x}}$. So like we covered in class the exact solution for the given $\boxed{y' = -\lambda y, \; y(x_0) = y_0}$ is as

(4) $\boxed{y_m(x) = A \cdot e^{-\lambda x},}$ so

$y_m(x) = Ae^{-5x}$ & by $y(0) = 1 \Rightarrow A = 1 \Rightarrow$

(5) $\boxed{y_m(x) = e^{-5x}}$ which matches exact sol-n in (3).

# Math548_hw9_Lazizbek

June 28, 2024

## 1 Lazizbek, Math 548, HW #9

#Homework-9

1. Solve dy/dx = y – x with y(0) =1 using the linear multi-step method

yi+1 +(3/2)yi -3yi-1 +(1/2)yi-2 = 3hfi.

First, find the exact solution for the given problem. Then, make use of the exact solution for any additional starting values in order to apply the linear multi-step method. Compare your numerical solution against the exact solution. Explain your observation.

2. For the initial value problems given below, find the solutions using the Picard iteration method.

(i) dy/dx = x y with y (0) = 1, x [0,1].

(ii) dy/dx = 2(y+1) with y (0) = 0, x [0,1].

3. A numerical method known as the Runge-Kutta method is given as ym+1 = ym + (h/6) (F1 + 4F2 + F3), where F1 = f(ym), F2 = f (ym + 0.5hF1) and F3 = f (ym – hF1 + 2hF2).Find the order of this method. Using this method, find the exact discrete solution of the initial value problem dy/dx = - 5y with y (0) = 1

## 2 Problem 1

[ ]:
```
import pandas as pd

# Define the initial conditions
x0_initial = 0
y0_initial = 1

# Step sizes to be used in the method
hlist = [1/(2**4), 1/(2**3), 1/(2**2), 1/(2**1), 2, 3]

# Function to calculate the exact solution
def exact_solution(x):
    return (x + 1)

# Outer loop to iterate over different step sizes
for h in hlist:
```

```
    # Re-initialize the initial conditions for each h
    x0 = x0_initial
    y0 = y0_initial
    xlist = [x0]
    ylist = [y0]
    results = []

    # Compute the first step using the exact solution
    x1 = x0 + h
    y1 = exact_solution(x1)
    xlist.append(x1)
    ylist.append(y1)
    exact_y1 = exact_solution(x1)
    error_y1 = abs(y1 - exact_y1)

    # Compute the second step using the exact solution
    x2 = x1 + h
    y2 = exact_solution(x2)
    xlist.append(x2)
    ylist.append(y2)
    exact_y2 = exact_solution(x2)
    error_y2 = abs(y2 - exact_y2)

    # Add initial steps to the results
    results.append([x0, y0, x1, y1, x2, y2, exact_y2, error_y2])

    # Inner loop to perform the iterations starting from the third step
    while x2 < 40:  # Continue until x2 reaches or exceeds 1
        y3 = (-3*y2)/2 + 3*h*(y2 - x2) + 3*y1 - (y0/2)
        x3 = x2 + h
        xlist.append(x3)
        ylist.append(y3)
        exact_y3 = exact_solution(x3)
        error_y3 = abs(y3 - exact_y3)
        # results.append([x2, y2, x3, y3, x3, exact_y3, error_y3])

        # Update the previous points for the next iteration
        x0 = x1
        y0 = y1
        x1 = x2
        y1 = y2
        x2 = x3
        y2 = y3
        results.append([x0, y0, x1, y1, x2, y2, exact_y3, error_y3])

    # Create a DataFrame from the results
```

```
    df = pd.DataFrame(results, columns=['x0', 'y0', 'x1', 'y1', 'x2', 'y2',␣
↪'Exact y2', 'Absolute Error'])

    # Print the DataFrame for the current h
    print(f"Results for h = {h}:")
    print(df)
    print("\n")

    # Print the final lists of x and y values for each h in an aligned manner␣
↪(2 rows format)
    # print(f"Final numerical x and y values for h = {h}:")
    # combined_results = pd.DataFrame([xlist, ylist], index=['x', 'y'])
    # print(combined_results)
    # print("\n")
```

```
Results for h = 0.0625:
          x0       y0       x1       y1       x2       y2  Exact y2  \
0     0.0000   1.0000   0.0625   1.0625   0.1250   1.1250    1.1250
1     0.0625   1.0625   0.1250   1.1250   0.1875   1.1875    1.1875
2     0.1250   1.1250   0.1875   1.1875   0.2500   1.2500    1.2500
3     0.1875   1.1875   0.2500   1.2500   0.3125   1.3125    1.3125
4     0.2500   1.2500   0.3125   1.3125   0.3750   1.3750    1.3750
..       …        …        …        …        …        …         …
634  39.6250  40.6250  39.6875  40.6875  39.7500  40.7500   40.7500
635  39.6875  40.6875  39.7500  40.7500  39.8125  40.8125   40.8125
636  39.7500  40.7500  39.8125  40.8125  39.8750  40.8750   40.8750
637  39.8125  40.8125  39.8750  40.8750  39.9375  40.9375   40.9375
638  39.8750  40.8750  39.9375  40.9375  40.0000  41.0000   41.0000

     Absolute Error
0               0.0
1               0.0
2               0.0
3               0.0
4               0.0
..              …
634             0.0
635             0.0
636             0.0
637             0.0
638             0.0

[639 rows x 8 columns]


Results for h = 0.125:
         x0      y0      x1      y1      x2      y2  Exact y2  Absolute Error
```

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0 | 0.000 | 1.000 | 0.125 | 1.125 | 0.250 | 1.250 | 1.250 | 0.0 |
| 1 | 0.125 | 1.125 | 0.250 | 1.250 | 0.375 | 1.375 | 1.375 | 0.0 |
| 2 | 0.250 | 1.250 | 0.375 | 1.375 | 0.500 | 1.500 | 1.500 | 0.0 |
| 3 | 0.375 | 1.375 | 0.500 | 1.500 | 0.625 | 1.625 | 1.625 | 0.0 |
| 4 | 0.500 | 1.500 | 0.625 | 1.625 | 0.750 | 1.750 | 1.750 | 0.0 |
| .. | … | … | … | … | … | … | … | |
| 314 | 39.250 | 40.250 | 39.375 | 40.375 | 39.500 | 40.500 | 40.500 | 0.0 |
| 315 | 39.375 | 40.375 | 39.500 | 40.500 | 39.625 | 40.625 | 40.625 | 0.0 |
| 316 | 39.500 | 40.500 | 39.625 | 40.625 | 39.750 | 40.750 | 40.750 | 0.0 |
| 317 | 39.625 | 40.625 | 39.750 | 40.750 | 39.875 | 40.875 | 40.875 | 0.0 |
| 318 | 39.750 | 40.750 | 39.875 | 40.875 | 40.000 | 41.000 | 41.000 | 0.0 |

[319 rows x 8 columns]

Results for h = 0.25:

| | x0 | y0 | x1 | y1 | x2 | y2 | Exact y2 | Absolute Error |
|---|---|---|---|---|---|---|---|---|
| 0 | 0.00 | 1.00 | 0.25 | 1.25 | 0.50 | 1.50 | 1.50 | 0.0 |
| 1 | 0.25 | 1.25 | 0.50 | 1.50 | 0.75 | 1.75 | 1.75 | 0.0 |
| 2 | 0.50 | 1.50 | 0.75 | 1.75 | 1.00 | 2.00 | 2.00 | 0.0 |
| 3 | 0.75 | 1.75 | 1.00 | 2.00 | 1.25 | 2.25 | 2.25 | 0.0 |
| 4 | 1.00 | 2.00 | 1.25 | 2.25 | 1.50 | 2.50 | 2.50 | 0.0 |
| .. | … | … | … | … | … | … | … | |
| 154 | 38.50 | 39.50 | 38.75 | 39.75 | 39.00 | 40.00 | 40.00 | 0.0 |
| 155 | 38.75 | 39.75 | 39.00 | 40.00 | 39.25 | 40.25 | 40.25 | 0.0 |
| 156 | 39.00 | 40.00 | 39.25 | 40.25 | 39.50 | 40.50 | 40.50 | 0.0 |
| 157 | 39.25 | 40.25 | 39.50 | 40.50 | 39.75 | 40.75 | 40.75 | 0.0 |
| 158 | 39.50 | 40.50 | 39.75 | 40.75 | 40.00 | 41.00 | 41.00 | 0.0 |

[159 rows x 8 columns]

Results for h = 0.5:

| | x0 | y0 | x1 | y1 | x2 | y2 | Exact y2 | Absolute Error |
|---|---|---|---|---|---|---|---|---|
| 0 | 0.0 | 1.0 | 0.5 | 1.5 | 1.0 | 2.0 | 2.0 | 0.0 |
| 1 | 0.5 | 1.5 | 1.0 | 2.0 | 1.5 | 2.5 | 2.5 | 0.0 |
| 2 | 1.0 | 2.0 | 1.5 | 2.5 | 2.0 | 3.0 | 3.0 | 0.0 |
| 3 | 1.5 | 2.5 | 2.0 | 3.0 | 2.5 | 3.5 | 3.5 | 0.0 |
| 4 | 2.0 | 3.0 | 2.5 | 3.5 | 3.0 | 4.0 | 4.0 | 0.0 |
| .. | … | … | … | … | … | … | … | |
| 74 | 37.0 | 38.0 | 37.5 | 38.5 | 38.0 | 39.0 | 39.0 | 0.0 |
| 75 | 37.5 | 38.5 | 38.0 | 39.0 | 38.5 | 39.5 | 39.5 | 0.0 |
| 76 | 38.0 | 39.0 | 38.5 | 39.5 | 39.0 | 40.0 | 40.0 | 0.0 |
| 77 | 38.5 | 39.5 | 39.0 | 40.0 | 39.5 | 40.5 | 40.5 | 0.0 |
| 78 | 39.0 | 40.0 | 39.5 | 40.5 | 40.0 | 41.0 | 41.0 | 0.0 |

[79 rows x 8 columns]

Results for h = 2:

| | x0 | y0 | x1 | y1 | x2 | y2 | Exact y2 | Absolute Error |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1.0 | 2 | 3.0 | 4 | 5.0 | 5 | 0.0 |
| 1 | 2 | 3.0 | 4 | 5.0 | 6 | 7.0 | 7 | 0.0 |
| 2 | 4 | 5.0 | 6 | 7.0 | 8 | 9.0 | 9 | 0.0 |
| 3 | 6 | 7.0 | 8 | 9.0 | 10 | 11.0 | 11 | 0.0 |
| 4 | 8 | 9.0 | 10 | 11.0 | 12 | 13.0 | 13 | 0.0 |
| 5 | 10 | 11.0 | 12 | 13.0 | 14 | 15.0 | 15 | 0.0 |
| 6 | 12 | 13.0 | 14 | 15.0 | 16 | 17.0 | 17 | 0.0 |
| 7 | 14 | 15.0 | 16 | 17.0 | 18 | 19.0 | 19 | 0.0 |
| 8 | 16 | 17.0 | 18 | 19.0 | 20 | 21.0 | 21 | 0.0 |
| 9 | 18 | 19.0 | 20 | 21.0 | 22 | 23.0 | 23 | 0.0 |
| 10 | 20 | 21.0 | 22 | 23.0 | 24 | 25.0 | 25 | 0.0 |
| 11 | 22 | 23.0 | 24 | 25.0 | 26 | 27.0 | 27 | 0.0 |
| 12 | 24 | 25.0 | 26 | 27.0 | 28 | 29.0 | 29 | 0.0 |
| 13 | 26 | 27.0 | 28 | 29.0 | 30 | 31.0 | 31 | 0.0 |
| 14 | 28 | 29.0 | 30 | 31.0 | 32 | 33.0 | 33 | 0.0 |
| 15 | 30 | 31.0 | 32 | 33.0 | 34 | 35.0 | 35 | 0.0 |
| 16 | 32 | 33.0 | 34 | 35.0 | 36 | 37.0 | 37 | 0.0 |
| 17 | 34 | 35.0 | 36 | 37.0 | 38 | 39.0 | 39 | 0.0 |
| 18 | 36 | 37.0 | 38 | 39.0 | 40 | 41.0 | 41 | 0.0 |

Results for h = 3:

| | x0 | y0 | x1 | y1 | x2 | y2 | Exact y2 | Absolute Error |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1.0 | 3 | 4.0 | 6 | 7.0 | 7 | 0.0 |
| 1 | 3 | 4.0 | 6 | 7.0 | 9 | 10.0 | 10 | 0.0 |
| 2 | 6 | 7.0 | 9 | 10.0 | 12 | 13.0 | 13 | 0.0 |
| 3 | 9 | 10.0 | 12 | 13.0 | 15 | 16.0 | 16 | 0.0 |
| 4 | 12 | 13.0 | 15 | 16.0 | 18 | 19.0 | 19 | 0.0 |
| 5 | 15 | 16.0 | 18 | 19.0 | 21 | 22.0 | 22 | 0.0 |
| 6 | 18 | 19.0 | 21 | 22.0 | 24 | 25.0 | 25 | 0.0 |
| 7 | 21 | 22.0 | 24 | 25.0 | 27 | 28.0 | 28 | 0.0 |
| 8 | 24 | 25.0 | 27 | 28.0 | 30 | 31.0 | 31 | 0.0 |
| 9 | 27 | 28.0 | 30 | 31.0 | 33 | 34.0 | 34 | 0.0 |
| 10 | 30 | 31.0 | 33 | 34.0 | 36 | 37.0 | 37 | 0.0 |
| 11 | 33 | 34.0 | 36 | 37.0 | 39 | 40.0 | 40 | 0.0 |
| 12 | 36 | 37.0 | 39 | 40.0 | 42 | 43.0 | 43 | 0.0 |

### Explanation and Observations

*Explanation of the Method:*

The method implemented here involves iterating over different step sizes  to solve a given differential equation numerically and compare it to the exact solution. We used an initial value problem (IVP) with  (0)=1. The exact solution to this differential equation is given by:

y(x)=(x+1)

The iterative method updates the  values using the following scheme:

*Numerical Results:*

h=0.25 show that the numerical values  closely match the exact values at each step  , resulting in zero absolute error. This indicates that the method is stable and accurate for this step size.

Similarly, the results for  =3 also show that the numerical values  match the exact values at each step  , again resulting in zero absolute error. This is somewhat surprising given the general expectation that larger step sizes ( h>2) should often introduce numerical errors.

**Observations** *Stability:* The method appears to be highly stable and accurate for both small and large step sizes, as evidenced by the zero absolute error observed in the results. This indicates that the numerical scheme is correctly implemented and the update formula is robust.

*Comparison with Exact Solution:*

The numerical solution exactly matches the exact solution for all tested values of  . This perfect match suggests that the iterative scheme used here is either inherently stable for this particular differential equation or that the specific initial conditions and solution form lead to an exact match.

*Effect of Step Size  :*

Despite expectations that step sizes larger than 2 should introduce numerical errors, the results show that even with  =3, the numerical solution remains accurate. I could not find a reliable reason for this, I'm still attached to the idea of existance of a condition on h due to the fact that the given linear multistep method is explicit method, I assume there should be limitations on h then.

*Stability Condition:*

The calculated stability condition for  being  $< 2$ was not strictly necessary to achieve zero error. This suggests that the numerical method might have a wider range of stability than initially calculated or that the exact nature of the update formula inherently maintains accuracy.

*Conclusion*

The numerical method implemented demonstrates excellent stability and accuracy for the given differential equation across a range of step sizes. The zero absolute error in comparison to the exact solution highlights the robustness of the iterative scheme used. Further investigation could involve testing the method on more complex differential equations or those with non-linear exact solutions to further explore the method's applicability and limitations.

[ ]:
```
# !sudo apt-get install texlive-xetex texlive-fonts-recommended␣
↪texlive-plain-generic
```

[ ]:
```
# !jupyter nbconvert --to pdf /content/Math548_hw8_Lazizbek.ipynb
```